Discovery Report: Automating Secure Onboarding at Scale

Oct 16, 2026 · Jordan Steve Lopez Nguepi

Field	Detail
Lab	Automating Secure Onboarding at Scale (Security & Identity Management on AWS, Week 1)
Name	Jordan Nguepi
Date	16 October 2026
Client	Onward Logistics

1. What I found

The client's need

Onward Logistics needs every new engineer to get exactly the access their job requires, set up the same way each time, with a record of who granted it and when, so fast hiring does not quietly widen what people can do.

Solutions that exist

Solution	How it works	Trade off for this client
Manual console setup with a written checklist	An admin follows a runbook and clicks each user, group and policy together in the AWS console.	**Strength:** no new tooling, fast for one hire. **Weakness:** this is today's problem. Results still depend on who is on shift, and there is no review step before a change goes live.
Infrastructure as code: CloudFormation from Git, with role groups and	Access is written in a template, reviewed as a pull request and deployed by CloudFormation Git sync. Groups grant role access; a	**Strength:** identical every time, every change has a named author and approver, and the boundary holds even if someone over-grants later. **Weakness:** the team must learn

permissions boundaries	boundary on each user caps the most they can ever do.	templates and pull requests, and IAM users still carry long-lived passwords.
AWS IAM Identity Center with AWS Organizations	Engineers sign in through a central identity portal; permission sets map to roles, and service control policies (SCPs) cap whole accounts.	**Strength:** short-lived credentials, single sign-on and MFA, and SCPs even an account admin cannot bypass. **Weakness:** needs a multi-account Organizations setup, which is more than a startup at Onward's stage has, and it is outside this lab's required IAM-plus-CloudFormation scope.

What happens with no solution

Hand-built access drifts. Over time, some engineers end up with more power than their role needs and nobody can say who granted it. That costs money, customer trust and audit time.

- **Real example:** in 2019 an attacker took over credentials from a misconfigured server at Capital One. The server's IAM role had far broader permissions than it needed, which let the attacker read S3 data on more than 100 million people (Krebs on Security). The US regulator later fined the bank $80 million for weak cloud risk controls (OCC, Aug 2020).
- **Disruption to operations:** an on-call engineer who can launch or terminate instances could, by mistake, stop delivery tracking systems during a live incident or run up an unexpected compute bill.
- **Audit exposure:** without a change record, Onward cannot answer "who approved this, and when?" That slows audits and weakens trust with the customers and partners who rely on its systems.

Sources

- Permissions boundaries for IAM entities, AWS IAM User Guide
- Syncing stacks with source code stored in a Git repository with Git sync, AWS CloudFormation User Guide
- What we can learn from the Capital One hack, Krebs on Security (Aug 2019)
- OCC assesses $80 million civil money penalty against Capital One (Aug 2020)

2. The business value of my solution

My chosen approach

I chose infrastructure as code: one CloudFormation template in Git, deployed by Git sync, using role groups to grant access and permissions boundaries to cap it.

It fits Onward better than the other options for three reasons:

- **Same result every time.** Onboarding a new hire means copying one user block and opening a pull request. The new hire gets the same group, the same boundary and a new generated password, and nothing is written per person.
- **Every change has a name and a time.** Nobody can change access without a reviewed pull request, and CloudTrail records any change made outside that process.
- **The on-call guarantee survives mistakes.** AWS allows an action only if both the group policy and the boundary allow it, and an explicit deny always wins. Even if someone later attaches full admin access to the on-call engineer, they still cannot launch or terminate compute.

Manual setup cannot give these guarantees. Identity Center with Organizations is the better long-term goal, but it needs a multi-account setup Onward does not have yet.

Services I will use

Service	The job it does	Why this service, used this way
AWS CloudFormation (Git sync)	Builds every user, group, boundary and secret from a template stored in GitHub. A merged pull request is the only way to deploy.	Required by the client. It makes access repeatable, and the whole setup can be rebuilt from the repository.
AWS IAM groups with AWS managed policies	Grant day-to-day access by role: S3 full access for storage, EC2 full access for compute, EC2 read-only for on-call.	Managed policies mean no inline or per-person policies. Adding a hire never means writing new permissions.
IAM permissions boundaries	Set the most each role can ever do. The on-call boundary allows only Describe, Get and List, and explicitly denies Run, Start, Stop and Terminate.	This is the forward-looking cap the client asked for. It holds even if someone loosens the group's permissions later.

AWS Secrets Manager	Generates a random 24-character first-login password for each engineer. The template reads it through a dynamic reference, and the engineer must change it at first sign-in.	No plain-text or shared password ever appears in Git, the template or stack parameters.
AWS CloudTrail (with an S3 log bucket)	Records every IAM change across all regions, with log-file validation, in a bucket kept even if the stack is deleted.	Answers "who changed this, and when?" for the Head of Security, including changes made outside Git. Deployed in Frankfurt (eu-central-1), recording all regions.

Who benefits, and how I will know

Stakeholder	Benefit	How I would know
Head of Security	Can see exactly how access is defined and who changed it, in one place.	For each objective, a policy-simulator allowed/denied report per user, plus CloudTrail events tying each change to a person and a time.
Leadership	Hiring can speed up without adding security risk or admin effort.	Onboarding one engineer takes one approved pull request; setup time per hire drops from a manual session to minutes.
On-call engineers	Can check that a deployment landed without any risk of breaking production.	As `ec2-user-2`, launching or terminating an instance fails, even after AdministratorAccess is attached as a test.
Cloud and platform admins	No more hand-built access, and fewer "why can this person do that?" questions.	Drift detection shows the stack matches the template, and the whole setup rebuilds cleanly from the repository.

New engineers	Working access on day one, and a private first password.	Each engineer signs in, is forced to change their password, and can do their role's tasks immediately.

Limits and risks of my design

- **A full IAM admin can still remove a boundary.** CloudTrail records it and drift detection flags it, but nothing prevents it, and someone has to check the logs to notice. An AWS Organizations SCP would prevent it.
- **IAM users have long-lived passwords, and MFA is not enforced.** Moving to IAM Identity Center with MFA is the long-term fix.
- **Access covers every bucket or every instance in the account.** Once naming and tagging standards exist, access should be scoped by tag or by resource.
- **The process depends on Git reviews being real.** If branch protection is turned off or approvals become rubber stamps, the audit trail stays but the review step loses its value.